

Wentworth Smart Academic Planner

Final Design Document and Project Report

Senior Project | Wentworth Institute of Technology

Project Team	Ibukunoluwa Folajimi & Ben N. Le
Advisor	Professor Salem Othman
System Type	Web-based academic planning and schedule recommendation system
Primary Technologies	Python, FastAPI, Streamlit, SQLAlchemy, SQLite, PostgreSQL, pdfplumber, python-docx, OpenAI API

Table of Contents

1. Overview.....	3
2. Purpose.....	3
3. System Summary.....	4
3.1. Student Features.....	4
3.1.1. Transcript Processing.....	4
3.1.2. Degree Progress.....	4
3.1.3. Schedule Builder.....	4
3.1.4. Curriculum Planner.....	4
3.1.5. Data Persistence.....	5
4. Software, Hardware, and Networking Components.....	5
4.1. Software.....	5
4.1.1. Primary Software Components.....	5
4.2. Networking.....	5
4.3. Hardware.....	6
5. Pre-existing Components.....	6
5.1. Frameworks.....	6
5.2. Libraries.....	6
5.3. External Services.....	6
6. Important Design Decisions.....	7
6.1. Separation of Frontend and Backend.....	7
6.2. REST API Design.....	7
6.3. Transcript Parser Design.....	7
6.4. AI as an Assistant.....	7
6.5. Configuration Management.....	7
6.6. Database Persistence.....	8
7. System Diagrams.....	8
7.1. System Architecture Diagram.....	8
7.2. Use Case Diagram.....	9
7.3. Transcript Processing Workflow.....	10
8. Major Milestones.....	11
8.1. Initial Planning.....	11
8.2. Backend Development.....	11
8.3. Frontend Development.....	11
8.4. AI Integration.....	11
8.5. Testing.....	11
8.6. Documentation.....	12
9. Goals of the Project.....	12
9.1. Goals Successfully Achieved.....	12
9.2. Goals Partially Completed.....	12
9.3. Goals Not Completed.....	13
10. Unknowns and Challenges.....	13
11. Things We Wish We Knew Before Starting.....	13
12. Final Reflections.....	14
13. References.....	15
13.1. Programming Language and Frameworks.....	15
13.2. Database and Persistence.....	15
13.3. Data Processing Libraries.....	15
13.4. Artificial Intelligence.....	16
13.5. Deployment and Development Tools.....	16

1. Overview

The Wentworth Smart Academic Planner is a web-based companion application designed to enhance the existing academic planning and course registration process at Wentworth Institute of Technology. Rather than replacing the current LeopardWeb registration system, the application complements it by providing students with an intuitive environment for transcript analysis, degree progress tracking, course selection, and semester planning.

The planner closely mirrors the workflow and course selection experience of LeopardWeb, allowing students to become familiar with the registration process while planning their schedules. By integrating transcript parsing, prerequisite validation, schedule conflict detection, and AI-assisted curriculum recommendations into a single platform, the application enables students to evaluate multiple scheduling scenarios before registering for courses. This approach helps reduce manual effort, minimize planning errors, and improve confidence in academic decision-making while maintaining compatibility with Wentworth's existing registration workflow.

This version emphasizes that the Smart Academic Planner is a planning companion rather than a replacement for LeopardWeb, while clearly introducing its primary functionality and overall purpose.

2. Purpose

Academic planning often requires students to navigate multiple systems and resources, including unofficial transcripts, degree requirement documents, prerequisite information, and the LeopardWeb registration system. This fragmented process can be time-consuming and makes it difficult for students to determine which courses they have completed, which requirements remain, and which courses they should take in future semesters.

The purpose of the Wentworth Smart Academic Planner is to streamline this process by providing a centralized application that integrates transcript analysis, degree progress tracking, course selection, and AI-assisted curriculum planning. Rather than replacing LeopardWeb, the application serves as a companion planning tool with a familiar interface that allows students to evaluate their academic progress and explore different scheduling options before registration.

The project was inspired by our own experiences during the early years of our degree program, when meeting with academic advisors was often necessary to fully understand degree requirements and course sequencing. By leveraging curriculum information provided by the School of Computing and Data Science, we developed a system that helps students independently monitor their progress, verify prerequisite requirements, and make more informed academic planning decisions.

3. System Summary

3.1. Student Features

Students can:

- Log into the application
- Upload an unofficial transcript (PDF or DOCX)
- Automatically detect completed courses
- Manually add completed or transfer courses
- View completed and remaining degree requirements
- Save completed courses and elective-credit information
- Automatically reload saved academic progress during future logins
- View completed and remaining degree requirements
- Review eligible courses
- Build a semester schedule
- Detect scheduling conflicts
- Generate an AI-assisted curriculum map

3.1.1. Transcript Processing

Users can upload their unofficial transcripts from Leopardweb and have them uploaded for analysis. This aids in subsequent points due to having enough data processed within the transcripts to gather completed courses, thus allowing us to cross reference them with the academic calendar automatically.

3.1.2. Degree Progress

The data from either manually entered entries or the transcript allows for there to be a simple comparison between the data logged and the required degree courses gathered through the academic calendar (which was available publicly and disclosed similarly to the previously mentioned check-ins with advisors).

3.1.3. Schedule Builder

Once classes are picked in the drop down menus, they are displayed in a calendar view reminiscent of the registration portal, but with differences that hopefully make the user experience worth the separate application. An important distinction that was created early on was the concept that classes which directly conflict with each other (in terms of timeframe, as you obviously cannot take more than 1 class at the same time) will be flagged accordingly.

3.1.4. Curriculum Planner

An addition that was suggested early on in the development cycle was the incorporation of an AI feature, which the idea was quickly established for the ability to give a rough outline of the potential choices that were on offer for subsequent semesters. Unfortunately, we did not have access to more than the current semester, so the feature ended up being a suggestion for the

current semester, with implementation in place so that if there was available data of future semesters, it would take that into consideration with the subsequent plans.

3.1.5. Data Persistence

The application stores user accounts and academic progress in a relational database through SQLAlchemy. Saved information includes completed required courses, custom or transfer courses, and elective-credit totals. SQLite is used during local development, while PostgreSQL supports persistent storage in the deployed environment. When a user logs in, the FastAPI backend retrieves the associated record and returns the saved information to the Streamlit frontend. When the user saves updated progress, the backend commits the changes to the database.

4. Software, Hardware, and Networking Components

4.1. Software

A lot of the implementation was done using pre-installed programs and libraries that happened to correlate with the task at hand, and not much was installed during development. There is one notable exception to this, though, and that would be the OpenAI API, given the fact that none of us had previously used anything that required the API before.

4.1.1. Primary Software Components

- Python
- Streamlit
- FastAPI
- SQLAlchemy
- SQLite
- PostgreSQL
- Psycopg
- pdfplumber
- python-docx
- OpenAI API
- Git
- GitHub
- Visual Studio Code
- Neon
- Render

4.2. Networking

The application uses a client-server architecture. The Streamlit frontend communicates with the FastAPI backend through HTTP requests and JSON responses. The backend processes transcript uploads, degree-progress requests, scheduling operations, and curriculum-planning

requests. It also communicates with the OpenAI API when generating AI-assisted recommendations.

In the deployed version, the FastAPI backend connects securely to a hosted PostgreSQL database using a database connection string stored in an environment variable. This database connection allows user progress to persist across application restarts and redeployments.

4.3. Hardware

The application does not require specialized hardware. Development and local testing can be performed on a standard computer capable of running Python, Streamlit, FastAPI, and SQLite. For cloud deployment, Render hosts the application services, while Neon provides the PostgreSQL database. The database is hosted separately from the application so that user information remains available when the application is restarted or redeployed.

5. Pre-existing Components

Although the majority of the application was developed specifically for this project, several third-party libraries and frameworks were incorporated to reduce development time and improve reliability.

5.1. Frameworks

FastAPI provides routing, request handling, validation, and REST API support. Streamlit provides rapid frontend development. SQLAlchemy provides object-relational mapping, database sessions, queries, and transaction management.

5.2. Libraries

- pdfplumber
- python-docx
- pandas
- requests
- pydantic
- Python-dotenv
- SQLAlchemy
- psycopg

5.3. External Services

The OpenAI API was integrated for curriculum recommendation generation. Neon provides hosted PostgreSQL storage for persistent user data, while Render supports deployment of the application services. Using established libraries and hosted services reduced development time while improving maintainability, persistence, and deployment reliability.

6. Important Design Decisions

Several important architectural decisions influenced the overall quality and maintainability of the application.

6.1. Separation of Frontend and Backend

Instead of placing all application logic directly inside Streamlit, the project separates presentation from business logic.

Advantages include:

- easier testing
- modular development
- clearer responsibilities
- easier future deployment

6.2. REST API Design

Communication between frontend and backend occurs exclusively through REST API endpoints using JSON. This allows other applications to potentially reuse the backend services without modification.

6.3. Transcript Parser Design

Originally, transcript parsing relied heavily on fixed formatting assumptions. However, testing quickly revealed that PDF extraction frequently merged words, removed spaces, and wrapped course information across multiple lines. Rather than depending on formatting, the parser was redesigned to recognize patterns involving course codes, grades, and credit values. This decision greatly improved parser robustness.

6.4. AI as an Assistant

Rather than allowing AI to determine course eligibility, deterministic planning logic first evaluates all prerequisite requirements. Only after valid remaining courses have been identified does the AI produce semester recommendations. This prevents AI from recommending courses that violate prerequisite constraints.

6.5. Configuration Management

Sensitive configuration information is stored in environment variables rather than being hardcoded into the source code. These values include the OpenAI API key and the database connection string.

The `DATABASE_URL` environment variable determines which database the application uses. If it is not provided during local development, the system defaults to SQLite. In the deployed environment, it contains the PostgreSQL connection string. This approach improves security, deployment flexibility, and separation between source code and environment-specific configuration.

6.6. Database Persistence

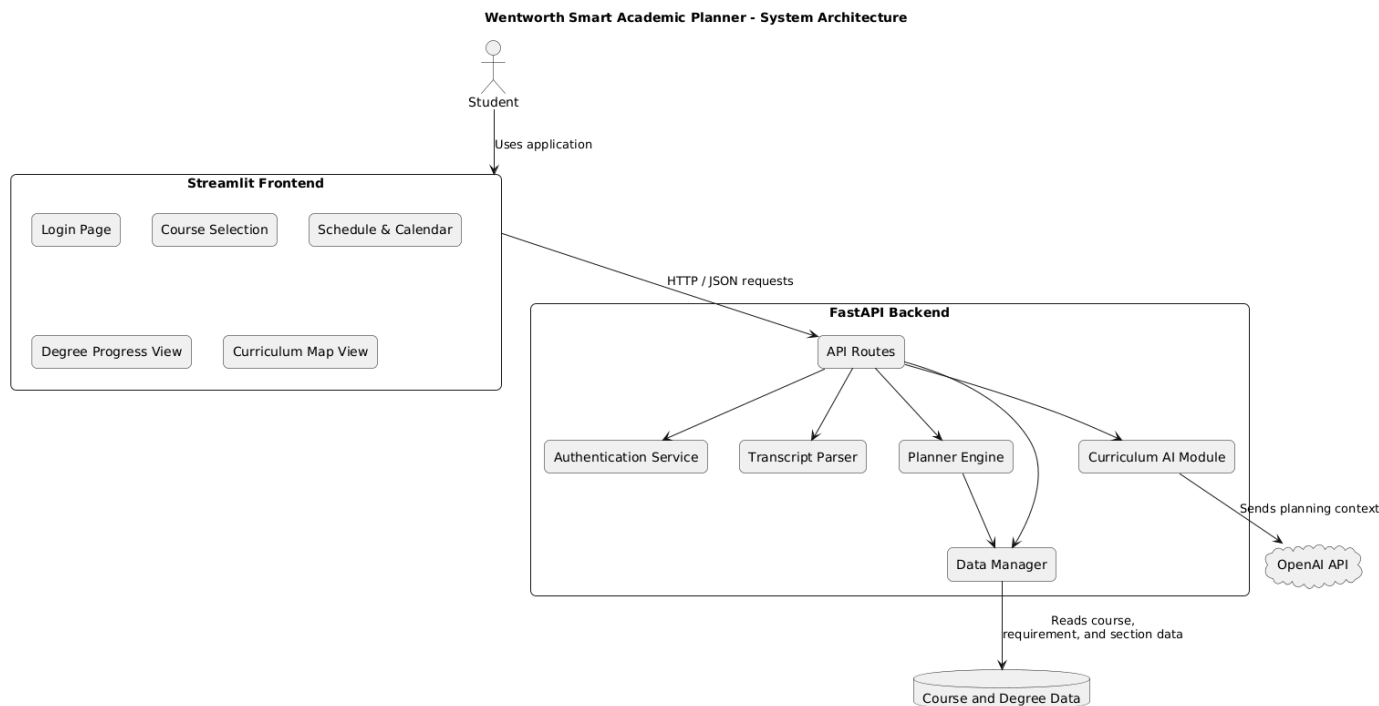
The original implementation stored user information in a CSV file. Although this approach was sufficient during early development, it was not appropriate for a deployed multi-user application because simultaneous updates could overwrite data, and local files might not persist after cloud redeployment.

The system was therefore redesigned to use a relational database through SQLAlchemy. SQLite is used for local development because it requires minimal configuration, while PostgreSQL is used in the deployed environment to provide persistent and reliable storage. Static academic reference data, such as course requirements and semester offerings, remains in CSV format because it changes less frequently and is maintained with the application.

7. System Diagrams

The following diagrams represent the major functional and technical aspects of the Wentworth Smart Academic Planner.

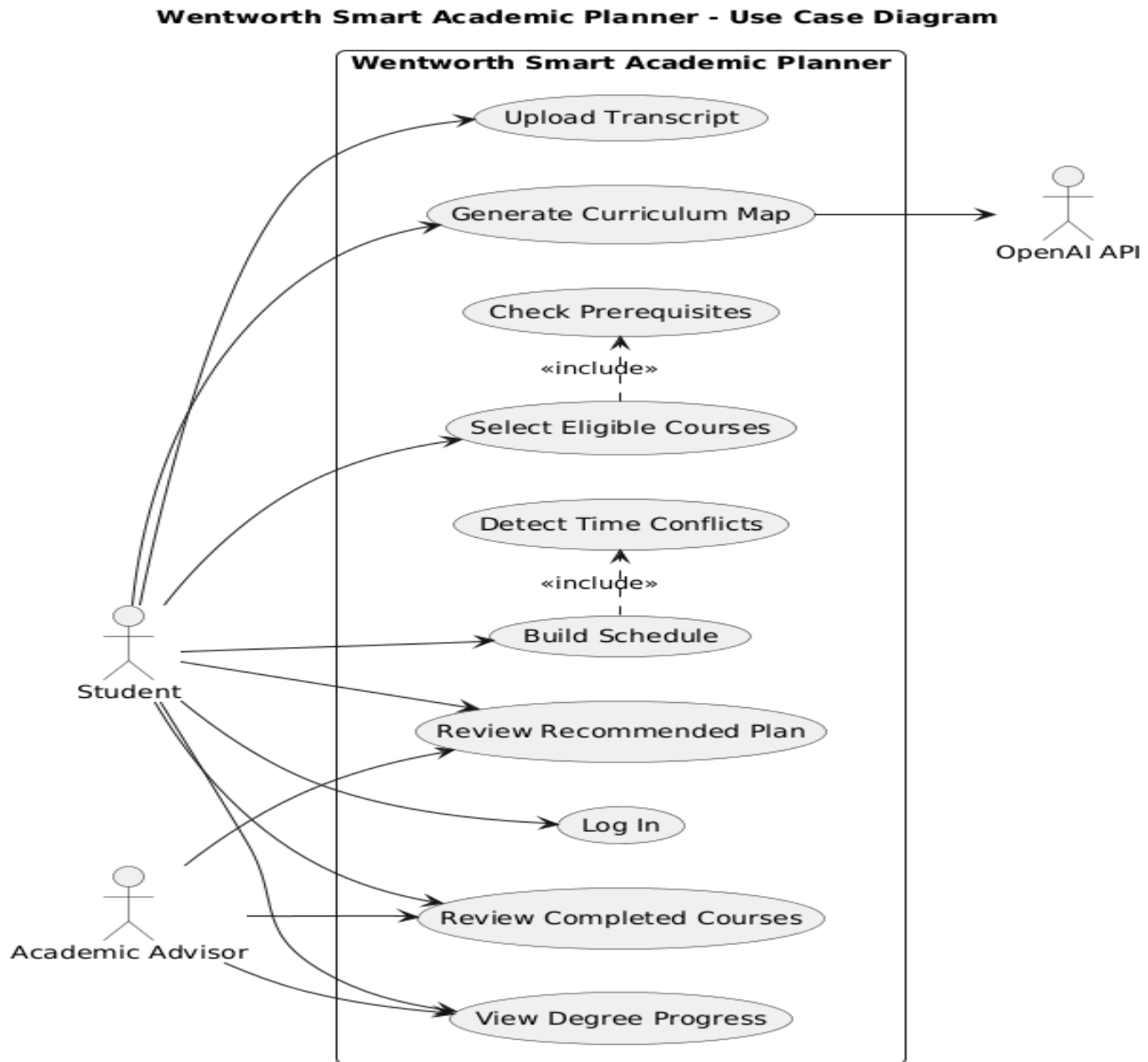
7.1. System Architecture Diagram



The System Architecture Diagram shows how students interact with the Streamlit frontend, which communicates with the FastAPI backend through HTTP and JSON. The backend coordinates authentication,

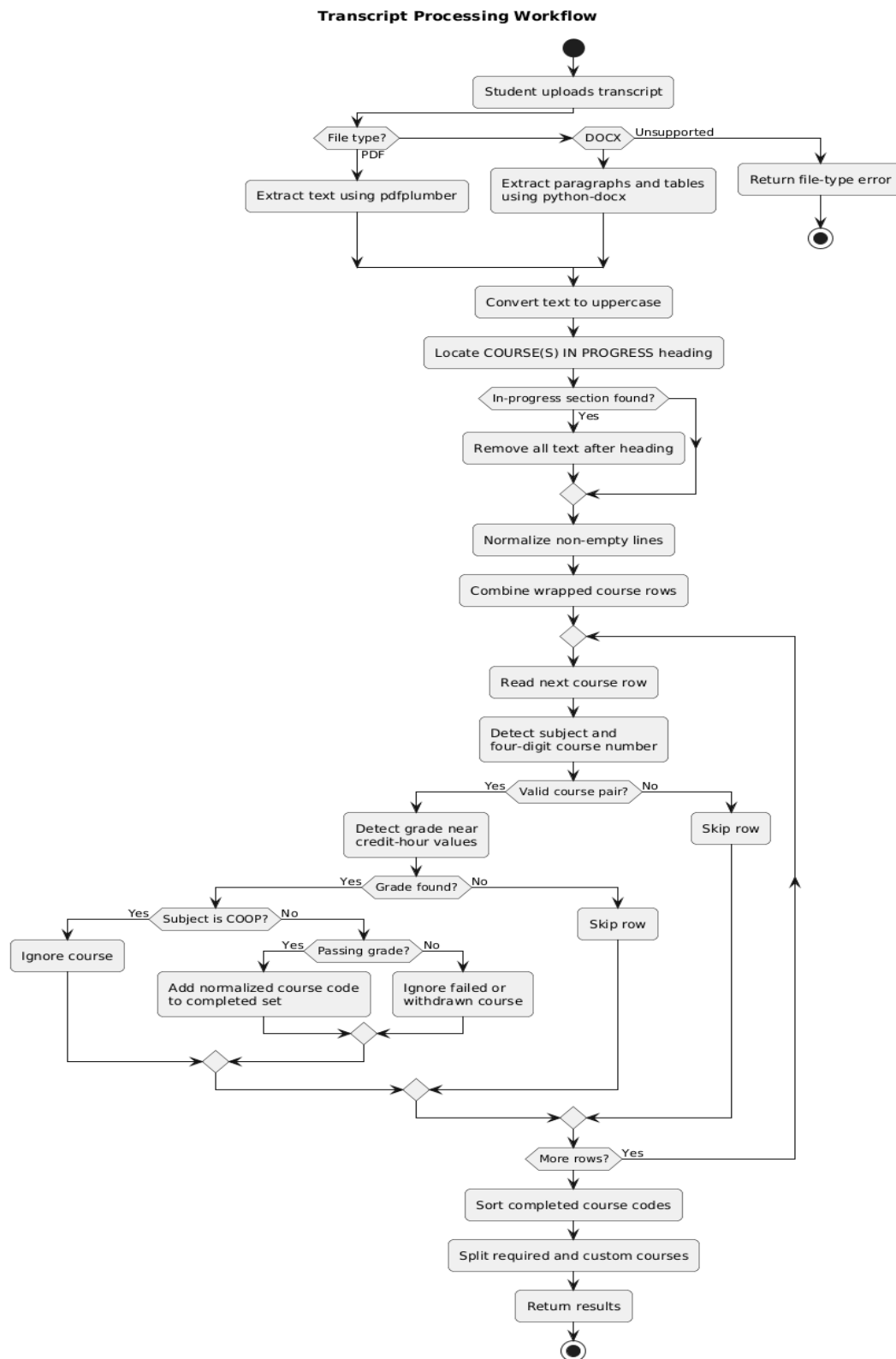
transcript parsing, degree planning, AI-assisted recommendations, static academic data, and persistent user storage. SQLAlchemy provides the database access layer; using SQLite locally and PostgreSQL in the deployed environment.

7.2. Use Case Diagram



The Use Case Diagram shows the main actions available to students. These include logging in, uploading a transcript, saving and later retrieving academic progress, reviewing completed courses, viewing degree progress, selecting eligible courses, building a schedule, detecting time conflicts, and generating a curriculum map.

7.3. Transcript Processing Workflow



The Transcript Processing Workflow shows how an uploaded transcript is converted into a list of completed courses. It includes file validation, text extraction, removal of in-progress courses, reconstruction of wrapped course rows, grade detection, and exclusion of failed, withdrawn, and co-op courses.

8. Major Milestones

Development progressed incrementally throughout the semester.

Major milestones included:

8.1. Initial Planning

- Project proposal approved
- Technology stack selected
- Repository initialized
- Development responsibilities assigned

8.2. Backend Development

- FastAPI project initialized
- API routes implemented
- Transcript parser created
- Degree planner implemented
- SQLAlchemy database layer implemented
- User database model created
- SQLite local persistence configured
- PostgreSQL deployment support added
- CSV user records migrated to the database

8.3. Frontend Development

- Streamlit interface completed
- Login interface created
- Course selection page implemented
- Schedule builder completed

8.4. AI Integration

- OpenAI API integrated
- Curriculum planning prompts refined
- AI recommendations tested

8.5. Testing

- Transcript parser tested using multiple unofficial transcripts
- Edge cases identified
- Schedule conflict detection validated
- Final debugging completed
- User save-and-reload behavior validated
- Database persistence tested across application restarts
- User-session isolation tested
- Database migration tested

8.6. Documentation

- Database migration documentation
- Environment configuration instructions
- Deployment configuration documentation
- Final report
- User manual
- Developer manual
- Testing report
- Final presentation

9. Goals of the Project

9.1. Goals Successfully Achieved

The major goals established during the proposal phase were successfully completed.

These include:

- Upload unofficial transcripts
- Parse PDF transcripts
- Parse DOCX transcripts
- Detect completed courses
- Ignore failed courses
- Ignore withdrawn courses
- Ignore co-op courses
- Ignore courses currently in progress
- Save completed required courses
- Save custom and transfer courses
- Save elective-credit totals
- Reload saved progress during later sessions
- Store user progress in a relational database
- Support SQLite for local development
- Support PostgreSQL for cloud deployment
- Calculate remaining requirements
- Recommend eligible courses
- Detect scheduling conflicts
- Generate AI-assisted curriculum plans
- Provide a responsive web interface

9.2. Goals Partially Completed

Some goals require additional development before production deployment.

Examples include:

- Production-level deployment hardening, monitoring, automated backups, and institutional security controls
- Support for every possible transcript format

9.3. Goals Not Completed

The following features were intentionally postponed:

- Integration with Wentworth's official registration system
- Automatic synchronization with Banner or DegreeWorks
- Student authentication using institutional login
- Expanded scope of courses, data was not available publicly for this purpose.

10. Unknowns and Challenges

The greatest challenge involved transcript parsing. Although transcripts appeared visually identical, extracted text frequently differed dramatically.

Examples included:

- missing spaces
- merged course titles
- wrapped rows
- inconsistent section headings

Several iterations of parser improvements were required before reliable results were obtained. Another challenge involved balancing AI recommendations with deterministic prerequisite rules. Allowing unrestricted AI recommendations occasionally produced academically invalid schedules. This was solved by validating eligibility before sending planning information to the AI. Deployment also presented challenges due to environment configuration, API keys, and dependency management.

Another challenge involved replacing CSV-based user storage with a relational database. The original CSV implementation was straightforward but did not provide reliable multi-user persistence in a cloud environment. The migration required creating SQLAlchemy models, managing database sessions, updating authentication and save functions, configuring environment-specific database URLs, and preserving existing user data through a migration script. Additional care was required to prevent Streamlit session-state values from one user from appearing when another user logged in.

11. Things We Wish We Knew Before Starting

At least on a personal level, Python and Streamlit knowledge was not mastered before the production of this, and many of the initial weeks were wasted having to learn and apply the knowledge that was required to set up the front-end of the program. There is perhaps more that can be desired, but the end result was well enough for the project and was a valuable learning

experience. Perhaps a different language or implementation would have been better for a more flexible customer-focused setup, but by the time the consideration crossed either of our minds we were too invested in it to give up weeks worth of progress.

Building on that experience, we also identified several other lessons that would have made development more efficient. We underestimated the complexity of PDF parsing; although reading a PDF initially seemed straightforward, extracting reliable, structured transcript data became one of the most time-consuming parts of the project. We also realized the importance of designing REST APIs more thoroughly before implementation, as several endpoints required refactoring as new requirements emerged. Additionally, prompt engineering demanded far more experimentation than expected. Producing useful curriculum recommendations required numerous iterations before the AI consistently generated practical semester plans. Finally, we learned the value of maintaining documentation throughout development instead of postponing it until the project's conclusion. Incrementally updating the documentation would have simplified the final submission process and ensured that the implementation remained aligned with the written design.

We also wish we had selected a persistent database earlier in the project. CSV storage was initially convenient, but it introduced limitations for user-specific data, cloud deployment, and concurrent updates. Designing the database schema and persistence layer earlier would have reduced refactoring near the end of development.

12. Final Reflections

The project ended up being a lot more of a learning experience than a polished product, but the experience was absolutely valuable, with python being preferred for more and more things and having its own niche for potential future projects down the line. There was a lot of points where at least for the front end, there could be improvements by swapping to react JS for a more polished and free-form front-end design instead of sticking to the more rigid design standards that streamlit expects (a particular qualm I had with it is the fact that you had to hardcode the page swapping sidebar button out of the site entirely in order to force page design through the buttons).

Despite these challenges, the project successfully achieved its primary objectives. The final application provides an integrated environment for transcript analysis, degree planning, course selection, and AI-assisted curriculum planning. Beyond the completed software, the project significantly strengthened our skills in software engineering, REST API development, frontend/backend integration, document parsing, and AI-assisted application development. With additional time, we would further enhance the system by integrating it with Wentworth's official academic systems, expanding transcript compatibility, strengthening authentication, normalizing course records into additional relational tables, adding automated database backups, and deploying a more robust production-ready cloud version.

13. References

13.1. Programming Language and Frameworks

[1] Python Software Foundation, “Python 3 Documentation,” 2026. [Online]. Available: <https://docs.python.org/3/>. [Accessed: Jul. 27, 2026].

[2] S. Ramírez, “FastAPI Documentation,” 2026. [Online]. Available: <https://fastapi.tiangolo.com/>. [Accessed: Jul. 27, 2026].

[3] Streamlit Inc., “Streamlit Documentation,” 2026. [Online]. Available: <https://docs.streamlit.io/>. [Accessed: Jul. 27, 2026].

13.2. Database and Persistence

[4] SQLAlchemy Authors, “SQLAlchemy Documentation,” 2026. [Online]. Available: <https://docs.sqlalchemy.org/>. [Accessed: Jul. 27, 2026].

[5] SQLAlchemy Authors, “SQLAlchemy ORM Quick Start,” 2026. [Online]. Available: <https://docs.sqlalchemy.org/orm/quickstart.html>. [Accessed: Jul. 27, 2026].

[6] PostgreSQL Global Development Group, “PostgreSQL Documentation,” 2026. [Online]. Available: <https://www.postgresql.org/docs/current/>. [Accessed: Jul. 27, 2026].

[7] Python Software Foundation, “sqlite3—DB-API 2.0 Interface for SQLite Databases,” 2026. [Online]. Available: <https://docs.python.org/3/library/sqlite3.html>. [Accessed: Jul. 27, 2026].

[8] Psycopg Development Team, “Psycopg 3 Documentation,” 2026. [Online]. Available: <https://www.psycopg.org/psycopg3/docs/>. [Accessed: Jul. 27, 2026].

[9] Neon Inc., “Neon Documentation,” 2026. [Online]. Available: <https://neon.com/docs/>. [Accessed: Jul. 27, 2026].

13.3. Data Processing Libraries

[10] J. Levine, “pdfplumber Documentation,” GitHub, 2026. [Online]. Available: <https://github.com/jsvine/pdfplumber>. [Accessed: Jul. 27, 2026].

[11] Python Open XML Community, “python-docx Documentation,” 2026. [Online]. Available: <https://python-docx.readthedocs.io/>. [Accessed: Jul. 27, 2026].

[12] The pandas Development Team, “pandas Documentation,” 2026. [Online]. Available: <https://pandas.pydata.org/docs/>. [Accessed: Jul. 27, 2026].

[13] Python Software Foundation, “Requests Documentation,” 2026. [Online]. Available: <https://requests.readthedocs.io/>. [Accessed: Jul. 27, 2026].

[14] Pydantic Services Inc., “Pydantic Documentation,” 2026. [Online]. Available: <https://docs.pydantic.dev/>. [Accessed: Jul. 27, 2026].

13.4. Artificial Intelligence

[15] OpenAI, “OpenAI API Documentation,” 2026. [Online]. Available: <https://platform.openai.com/docs/>. [Accessed: Jul. 27, 2026].

13.5. Deployment and Development Tools

[16] Render Services, Inc., “Render Documentation,” 2026. [Online]. Available: <https://render.com/docs>. [Accessed: Jul. 27, 2026].

[17] GitHub, Inc., “GitHub Documentation,” 2026. [Online]. Available: <https://docs.github.com/>. [Accessed: Jul. 27, 2026].

[18] Microsoft, “Visual Studio Code Documentation,” 2026. [Online]. Available: <https://code.visualstudio.com/docs>. [Accessed: Jul. 27, 2026].